PLANRS ACADEMY

Interactive Fiction, Sequential Grammar, and Branching Logic

Coding Your Choice:

Building a Secret Cave Adventure with Python If-Else

Master Lesson Guide & Code Terminal Companion • Grade 3

Total Duration	Logic Structures	Primary Keywords
30 Minutes	**Conditional Branching**	**if / else / input**

Lesson Design & Pedagogical Strategy

This master manual details the production layout, interface code blocks, and educational parameters for delivering the introductory branching logic module to Grade 3 school students. At this primary milestone, computer science education becomes highly impactful when children move away from passive reading and realize text statements can act as dynamic gateways that react to user choices.

Pedagogical Model

To reduce syntax frustration and focus purely on relational concepts, advanced structural items (such as explicit multi-conditional loops, nested chains, variable arithmetic operations, data format transformations, or nested boolean operators) are completely banned. The module uses an observational concrete-pictorial-abstract (CPA) structure. By connecting story decisions directly to terminal outputs on a split-screen canvas, students build an intuitive understanding of branching pathways before writing code manually.

Target Learning Objectives

- **LO 1 (Cognitive/Recall):** Identify, type, and state the exact behavior of two fundamental Python interaction commands: ***print()*** and ***input()***.
- **LO 2 (Conceptual):** Explain how a computer checks a text response to fork a program path using standard conditional blocks.
- **LO 3 (Application):** Connect statements sequentially to construct a working, two-way branching text adventure game on a digital terminal box.
- **LO 4 (Affective):** View text programming as an imaginative creative space, linking structural code paths to real-world decision-making scenarios.

Syntax Validation Safeguard

To support 8-year-old learners, syntax errors must never be treated as flat failures. Frame a missing colon or bad indentation space as a helpful tip: *"The checker spotted a tiny missing gatepost! Let's adjust our formatting to let the logic flow,"* keeping the environment encouraging and stress-free.

The Story Hook: The Lost Key of Ajanta Caves

The lesson starts with an engaging regional family narrative set during an exploration trip, establishing a practical problem that frames coding choices as a magical tool for story building.

> **The Setting:** Young Ananya and her tech-savvy cousin Kabir are exploring a hidden garden ruins path near an ancient historic fort site. The stone masonry is covered with thick green ivy vines.

Discovering the Carved Gateway

As they push aside the thick vines, they uncover a heavy iron gateway with two mysterious symbols carved into its center plate:

> **Ananya says:** "Oh look, Kabir! A locked doorway with a left path symbol and a right path symbol. We don't have a physical key to open it. How can we ever figure out what secrets are waiting on the other side?"

Kabir smiles encouragingly, opens up a laptop on a flat rock surface, and reveals a superpower: "We don't need a heavy brass key, Ananya! We can build a magical text gateway using Python code. We will write a choose-your-own-adventure story game where the computer waits for a player's typed word, and opens the door if they choose the correct path!"

The Analogy: Conditional Branches as a Fork in the Road

Before introducing code keywords, the lesson anchors conditional logic in physical choices that students already know from family trips.

Navigating a Forest Trail Split

Imagine you are hiking along a mountain trail during a family holiday at a scenic hill station like Mahabaleshwar or Ooty. Suddenly, the path splits into two distinct paths, requiring an immediate decision.

The Mountain Trail Choice	The Python Logic Setup	The Execution Outcome
IF you choose the left path:	`if path == "left":`	Walk to the beautiful waterfall view.
ELSE (Otherwise, if not left):	`else:`	Take the right path to visit the old temple ruins.

The computer checks the player's typed input against our rule conditions, guiding them down the matching story track instantly. It cannot travel down both paths at once; it chooses a single path based on the user's input.

Command Phase 1: Speaking through Print

This section explores how to program our story to speak out loud, examining the exact syntax structure required to display text descriptions on screen.

The Megaphone Command

To display text messages on the user terminal screen, we write the command name followed by our story description enclosed inside quotation marks and brackets:

```
print("You stand inside a mysterious dark cave entrance!")
```

Analyzing the Terminal Canvas Output

The instant this line of script executes, the words appear cleanly inside the player's output box. The quotation marks act as protective walls, letting the computer know these exact words belong together as a story description for the player to read.

> **Visual Mapping Guideline**
>
> The split-screen workspace view must connect the running text script on the left directly to the resulting text box lines on the right, highlighting items as they execute to make logic paths clear.

Command Phase 2: Gathering Input Responses

Next, we examine the input command, focusing on how it prompts the computer to wait patiently for a player's typed answer.

The Empty Collection Box

To let players type answers into our game, we use the input function, creating a placeholder variable to store their choice:

```
choice = input("Type left or right: ")
```

This command acts as an empty collection box. It pauses the story execution line and displays a blinking cursor, waiting patiently for the human player to type a word and press enter before passing the text down the logic path.

print("text")

Displays descriptions out loud on the screen, letting the player know exactly what environment or room they have entered.

input()

Pauses execution and displays a blinking cursor, capturing the player's typed answer to guide them at logic crossroads.

Interactive Story Architecture: Branching Networks

Once students master print statements and inputs, they can combine them with conditional blocks to construct choose-your-own-adventure pathways.

The Logic of Branching Narratives

Interactive text fiction is built by linking different text modules together using conditional checkpoints. By coding our digital system to evaluate player inputs, we can build custom storytelling frameworks, turning basic code lines into an immersive game landscape.

LIVE CODE-TO-CONSOLE LAB SIMULATION

Active Interface Mapping: The workspace projects text code on the left and a live console output on the right. When a user types "left", an on-screen animation shows the answer floating into a fork-in-the-road box to select the matching path.

The Hidden Treasure Room Challenge

Now, students step into the role of game developers, taking on the challenge of arranging code blocks sequentially to build a safe path through a text maze puzzle.

The Adventure Blueprint

Our game track requires a description of a dark cave room, an input prompt for the player, and a two-way branching choice. To build this successfully, our code blocks must be arranged in a clear sequence:

The Target Code Layout:

1. `print("You see a glowing gold door!")` -> Describe setup
2. `door_choice = input("Open or walk away? ")` -> Capture user choice
3. `if door_choice == "open":` -> Establish path check

This challenge demonstrates how sequential statements and conditional crossroads work together, helping children explore computational logic through creative storytelling.

Hangar Test 1: The Missing Gatepost Error

To help students master syntax rules, we walk through common formatting bugs to see how small omissions impact communication with a computer.

The Missing Colon Script

Imagine a student sets up a branching condition but leaves out the essential colon symbol *(:)* at the end of the line:

```
if path == "left"
    print("You found a treasure chest!")
```

Analyzing the Halting Bug

Without the colon symbol at the end of the statement, the computer cannot process the branch. A warning flashes on the console, letting the student know the checker spotted a missing gatepost and prompting them to add a colon to let the logic flow safely.

Hangar Test 2: The Spacing Indentation Error

This section explores indentation spacing, checking if students can identify the nesting required to group path reactions inside conditional blocks.

The Misaligned Spacing Script

Let's examine a script layout where the path reaction descriptions are pushed completely flat against the left margin, missing the required indent spacing:

```
if path == "left":
print("You fall into a deep mud pit!")
```

Reading the Spacing Warning

Because the print statement isn't indented, the computer can't figure out that this reaction belongs inside the left path choice. The script balances jumble up, and a warning appears prompting the student to add indentation spacing to separate the pathways clearly.

Hangar Test 3: Compiling the Safe Pathway

For our final test, we combine all our keywords and punctuation symbols correctly, arranging a complete working script with perfect indentation spacing.

The Complete Adventure Script

We lay out our printing megaphones, input collection boxes, colons, and indented pathways in our code file to build the game blueprint:

```
print("You enter a secret cavern ruins chamber!")
path = input("Type left or right: ")

if path == "left":
    print("You walk to a beautiful waterfall view!")
else:
    print("You discover an old historic temple shrine!")
```

Confirming the Game Output

With this balanced script, the game compiles perfectly. When a player types "left", the computer checks the condition and guides them safely to the waterfall scene, triggering a success celebration!

Data Synthesis: Story Logic Patterns in Review

This section summarizes our code laboratory findings, helping students see how keywords, colons, and spacing layout rules work together to guide story paths.

The Logic Interaction Profile

Comparing our three script arrangements side-by-side makes the computational patterns clear and easy for young minds to follow:

Script Format Arrangement	Terminal Screen Behavior	Computer Interpretation Status
1. Missing Colon Symbol	**Syntax Error**	The script halts at the gatepost omission, flashing a colon tip.
2. Flat Left Alignment	**Spacing Error**	The paths jumble together, prompting a spacing indent fix.
3. Indented Alternating Code	**Safe Story Branch**	Success! The game runs smoothly, switching paths based on choices.

Concept Review 1: Activating the Question Box

This assessment section uses targeted multiple-choice questions to verify that students can identify the correct function needed to capture player answers.

Review Question

Imagine you want your text adventure story to pause and wait for a human player to type their chosen path option into the console. Which Python command allows you to capture their text response?

SELECT THE CORRECT OPTION BUTTON

- A) print("path")
- B) input()
- C) else:
- D) if path == open

Correct Choice Pathway: Option B is correct! The input function serves as our empty collection box, pausing execution to gather the player's typed answer before moving forward down the script.

Concept Review 2: Spotting the Conditional Gateway

This second challenge checks if students can identify the required symbol that must always sit at the end of an if line statement.

Review Question

When writing an if statement crossroads in our adventure game script, what specific punctuation symbol must always sit at the end of that condition line to let the logic flow into the next path?

A) Question Mark (?)

B) Colon Symbol (:)

C) Plus Sign (+)

Correct Choice Pathway: Option B is correct! The colon symbol acts as our critical gateway post, marking the boundary of the check condition line and channeling the computer smoothly into the indented reaction steps.

Summary: Master Choice Logic Takeaways

As we wrap up our interactive fiction programming adventure, let's look back at the core coding concepts we've explored today about conditional branches and sequential storytelling.

Key Programming Discoveries

- **Crossroads Create Choice:** Computers use `if` and `else` conditional blocks to evaluate player input, forking a story down separate custom paths based on their response.
- **Functions Manage Interaction:** We use print megaphones to describe the landscape out loud, combined with input collection boxes to gather player text choices smoothly.
- **Formatting Keeps Tracks Clear:** Punctuation colons and indented indentation spacing are required grammar rules that group path reactions inside our conditional choices.

A Message for Our Story Code Builders

"By learning to connect text commands to branching choices, you've unlocked the core logic secrets that help game developers build vast digital worlds across the entire universe!"

The Dinner Flowchart Decision Quest!

Scientific discovery and code logic don't have to end in the classroom! You can continue your decision tracking journey at home with a fun flowcharting challenge.

YOUR HOME FLOWCHARTING QUEST

Create an if-else decision flowchart for dinner tonight! Take a blank sheet of paper and map out your choices into separate boxes using this exact logic flow:

- Draw a central choice box question: *"Is Mummy cooking paneer tonight?"*
- Draw a left branch arrow labeled **IF** (Yes) -> Path step: *"I will eat 3 hot rotis with dinner!"*
- Draw a right branch arrow labeled **ELSE** (No) -> Path step: *"I will eat 2 large bowls of dal rice!"*

Sharing Your Logic Map

Color your two choice pathways using different markers and explain your logic map to your family during dinner! Explaining your branching choices out loud is a fantastic way to show off your new programming skills.

Congratulations, Story Code Builders!

You have successfully completed your very first steps into conditional programming, unlocking the branching secrets that turn text commands into interactive adventure games!

Master Story Code Builder Milestone Achieved!

Curriculum Design Validation Framework

To maintain high educational standards, this master lesson companion manual matches the following design and primary computing metrics:

Curricular Linkage	100% compliant with primary computational thinking milestones and early sequential logic standards across national frameworks.
Cognitive Loading	Replaces complex variable type casting with intuitive, narrative-driven path scenarios and block analogies.
Context Mapping	Anchors text programming text lines to historic regional locations and daily domestic decision scenarios to maximize student retention.